Research Agent — Laptop-as-Server Deployment Plan

Run Chainlit locally on Windows and share it over HTTPS using a Cloudflare Quick Tunnel

Prepared: 10 October 2026

Repository: https://github.com/BGRAMPAGE/research_agent

1. Goal and topology

- Goal: host the app on your Windows laptop with no rented server and allow a recruiter/collaborator to access a demo URL.
- Topology: browser → Cloudflare Quick Tunnel temporary HTTPS URL → cloudflared → Chainlit on 127.0.0.1:8000 → local PostgreSQL/pgvector and optional local Ollama; hosted LLM/search APIs may still require their own quota/API key.
- This avoids server rental but is not costless in the literal sense: laptop electricity, internet, hardware wear, and availability remain your responsibility.

2. Stack and public-exposure safety

- Windows 10/11 + PowerShell, Python virtual environment, Chainlit, LangGraph, current repository, PostgreSQL/pgvector, optional Ollama, cloudflared Quick Tunnel.
- A Quick Tunnel gives a random trycloudflare.com URL suitable for temporary testing; it is not a stable named URL. A named published-application hostname normally requires a domain managed by Cloudflare. Do not assume you can get a permanent custom URL without a domain.
- Treat the tunnel as public internet exposure. A random URL is not authentication. Enable app authentication/access control where possible and use rate limits. Do not expose PostgreSQL, Ollama, SSH, RDP, or arbitrary ports; tunnel only the Chainlit web port.
- Keep Windows Defender/firewall enabled. Use a least-privilege database account. Do not expose confidential papers or private documents without reviewing access controls and retention.

3. Start locally

- 1. Open PowerShell in the project directory, for example `cd C:\Users\<you>\Desktop\10_Months\open_s_p\self_learning_agent`.
- 2. Activate the environment: `.\.venv\Scripts\Activate.ps1`. If PowerShell blocks activation, use an appropriate current-user solution rather than globally weakening execution policy.
- 3. Pull and inspect the desired version: `git pull origin main` then `git log -1 --oneline`.
- 4. Install dependencies according to the repository's documented dependency file; use `pip install -r requirements.txt` only if requirements.txt is the actual dependency source.
- 5. Create local `.env` from `.env.example`. Keep secrets out of Git and verify .gitignore excludes .env.
- 6. Confirm PostgreSQL is running, pgvector is enabled, schema/tables exist, and paper chunks are indexed. Test a known retrieval query.
- 7. If using Ollama, run `ollama list` and confirm the configured model is downloaded and works. For hosted APIs, verify keys/quota.
- 8. Start Chainlit using the project's documented command. A common command is `chainlit run app.py --host 127.0.0.1 --port 8000`; verify the pinned version and app configuration.
- 9. Open http://127.0.0.1:8000 locally and test both profiles before exposing it.

4. Create the temporary HTTPS tunnel

- 1. Install cloudflared for Windows using Cloudflare's official instructions.
- 2. Keep Chainlit running at http://127.0.0.1:8000.
- 3. Open a second PowerShell window and run `cloudflared tunnel --url http://127.0.0.1:8000`.
- 4. Wait for the generated `https://<random-name>.trycloudflare.com` URL.
- 5. Test the URL from a phone on mobile data or another network. Verify both profiles, WebSocket/progress updates, citations, and expected access controls.
- 6. Keep both PowerShell processes running. If the app or tunnel stops, the public site becomes unavailable.
- 7. Stop the tunnel with Ctrl+C when finished. The Quick Tunnel URL is temporary and may change on restart. Do not use router port forwarding as a casual workaround.

5. Stable URL options

- Quick Tunnel is easiest when you do not own a domain, but its random URL is temporary. Restarting cloudflared may give a different URL.
- If you later obtain a domain and manage it through Cloudflare, create a named tunnel and map a hostname to http://localhost:8000. Review current Cloudflare Access requirements if adding identity-based access policies.
- Do not treat an obscure URL as a security boundary. For public demos, authentication and request limits are strongly recommended, especially if the app can spend hosted-model API quota.

6. Reliability and performance

- Keep the laptop powered and connected. Sleep, Windows updates, reboots, Wi-Fi loss, and ISP outages interrupt the service. The laptop is a single point of failure.
- Local LLM inference and research runs compete for RAM/CPU/GPU. Limit concurrency, search iterations, output length, and document size. Verify model VRAM/RAM requirements instead of assuming the GPU can load every model.
- Back up the PostgreSQL data and original paper corpus. A local database alone is not a backup.
- Before sharing, verify app, database, model endpoint, and tunnel health. Do not leave a public demo running unattended indefinitely.

7. Troubleshooting and real-world fit

- 502 or connection error: confirm Chainlit is running on the same local port specified by cloudflared.
- Remote but not local failure: inspect tunnel logs, host/port configuration, WebSocket support, and app URL assumptions. Never open the database port to fix this.
- No Paper Library results: check DB URL, schema, vector extension, vector dimension, corpus ingestion, and embedding model consistency.
- Research hangs: inspect provider quota, search/model timeouts, network access, and graph progress updates.
- Good fit: portfolio demonstration, supervised walkthrough, small-group test, and local-model experiments. Not suitable as-is for 24/7 SaaS, high traffic, sensitive multi-tenant data, or uptime commitments.
- To move toward production, add authentication/authorization, rate limiting, backups, safe logs, security/dependency scanning, request timeouts, evaluation, and a recovery procedure.

8. Official documentation

- Cloudflare Tunnel setup: https://developers.cloudflare.com/tunnel/get-started/
- Cloudflare published application routing: https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/routing-to-tunnel/
- Cloudflare Tunnel routing concepts: https://developers.cloudflare.com/tunnel/concepts/routing/

Maintenance note

Before implementation, verify the exact repository commit and re-check provider pricing, free-tier quotas, terms, and security settings. Free-tier limits change.